standards compliant, and they provided few opportunities for customizing their output.

In an attempt to appeal to Windows developers, Web Forms introduced a large number of concepts to web development totally unique to the framework. While Web Forms is undeniably a hugely powerful framework, its high learning curve makes it a difficult starting point for anyone new to web development.

ASP.NET MVC was introduced in 2008, partly to provide a more *natural* web development experience, and as a result, it very quickly gained traction among intermediate and experienced ASP.NET web developers as well as developers more accustomed to using competing technologies, such as Ruby on Rails and Java Spring. ASP.NET MVC was also designed to solve several other problems advanced developers have with Web Forms, including a lack of testability, a need for clearer separation of concerns, the ability to extend the framework, and so on. A notable problem MVC did not solve, however, was making learning ASP.NET any easier; if anything, the MVC framework's concept count was higher than that of Web Forms. But the traditional mapping between filenames and URLs was just one casualty. Developers also had to consider the names of controller classes and their methods when building URLs as well as the names of view templates, which had to be placed in a specific location.

Microsoft attempted to solve the high barrier to entry for beginner developers by introducing another page-focused framework: ASP.NET Web Pages. This new framework restored the easy-to-reason-about one-to-one mapping between filenames and URLs. While it undoubtedly offered a simpler learning experience, Web Pages reintroduced the same issues ASP suffered from. A heavy reliance on the `dynamic` type resulted in a loss of strong typing in key parts of the framework, and the development model